

BACHELOR'S THESIS DEFENSE

# AI-ASSISTED MASTER PROMPT GENERATION CHALLENGES AND PROPOSED SOLUTIONS

---

Nguyen Thanh Tu | Student ID 10422080

17 August 2026

Supervisor 1: Assoc. Prof. Garcia Clavel Manuel

Supervisor 2: Dr. Nguyen Tuan Cuong

Computer Science, Vietnamese–German University

# Outline

Motivation & Problem

Research Questions & Contributions

Related Work

Methodology

Experiment 1: Traceability Evaluation

Experiment 2: Semantic Conflict Handling

Conclusion

## Motivation & Problem

---

# What Is a Master Prompt?



- Requirements evolve across prompts through additions, modifications, replacements, and removals.
- A master prompt captures the user's currently valid intent — not the entire conversation history.
- It consolidates the evolving requirements into one self-contained specification that can be reused across sessions, agents, or developers.
- It keeps only final active requirements and remove all unsupported details.

## DEFINITION

A **master prompt** is a single, self-contained specification of the user's currently valid intent.

# A Master Prompt Is Not Exactly a Conversation Summary

“Build a **single-page** portal homepage **as a static site using only plain HTML, CSS, and vanilla JavaScript (three files: index.html, style.css, app.js) — no frameworks or build step.** Layout & navigation: use a horizontal top navigation bar (no sidebar) **with a brand/logo on the left and the nav buttons on the right.** Include exactly four navigation buttons: Dashboard, Policies, History, Contact. **Clicking a button switches between four corresponding in-page views without reloading; the active button is highlighted.**”

**Error 1 — Unsupported details (extrinsic hallucination) (bold):** details introduced during consolidation that are not supported by the user’s prompt history.

**Error 2 — Inactive constraint (underlined):** the sidebar was removed earlier, but stating “no sidebar” not an active requirement in the ground-truth Master Prompt..



Claude · simple project · basic consolidation — AlignScore 0.086, 92% untraceable

# From Prompt History to a Master Prompt



- “Create a portal homepage.”
- “Add 3 navigation buttons: Home, Orders, Contact.”
- “Add sidebar navigation.”
- “Undo sidebar.” (undo)
- “Use top navigation instead.” (replace)
- “Change navigation buttons: Dashboard, Policies, History, Contact.” (replace)



consolidate



## Expected Master Prompt

“Create a portal homepage. Add top navigation. The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”

**A master prompt is not a summary of everything the user said — it is a specification of what is still valid at the end of the history.**

# Properties of a Reliable Master Prompt

- **Faithful** — every requirement is supported by the user's prompt history; unsupported additions or extrinsic hallucinations are excluded.
- **Current** — later modifications, removals, and replacements are reflected in the final specification.
- **Conflict-aware** — if a later instruction conflicts with a settled requirement without clearly requesting a revision, surface the conflict instead of silently changing the requirement.

## **Research Questions & Contributions**

---



# Research Questions

- **RQ1** — To what extent do AI-generated master prompts contain non-traceable content, including agent-added and outdated requirements, and how does consolidation instruction strictness affect this problem?
- **RQ2** — Does spec-guided consolidation produce a more traceable master prompt than single-pass consolidation of the raw prompt history?
- **RQ3** — When a later instruction semantically conflicts with a settled requirement, how can a spec-guided approach surface the conflict, trace both sides, and avoid resolving it without user input?

- **Requirement-level traceability evaluation** — a requirement-level method using atomic decomposition and entailment-based AlignScore to evaluate whether candidate master-prompt requirements are supported by the ground truth.
- **Spec-guided consolidation** — a maintained specification is updated throughout the prompt history rather than reconstructed only at the end.
- **Conflict-aware project handoff** — later instructions are checked against the settled specification; unclear conflicts are preserved for user resolution rather than silently resolved.

## Related Work

---

# Related Work

- **Mondal et al. (2024)** — *Enhancing User Interaction in ChatGPT* — combines sequences of prompts into one prompt; the main goal is interaction efficiency. Does not evaluate requirement-level traceability or later undo/replacement behavior.
- **AlignScore / FactScore / Claimify** — provide ideas for semantic support and atomic evaluation. Do not construct or maintain a master prompt.
- **Spec-driven development** — treats a specification as a maintained artifact. Does not directly evaluate unsupported or outdated requirements in an AI-generated master prompt.

# Closest Prior Work: Semantic Commit

- **Semantic Commit** checks new information for semantic conflicts before updating an intent specification and can suggest resolutions for user review.
- **My thesis:** studies the construction of a Master Prompt that remains traceable to the original prompt history.
- For unclear conflicts, the settled requirement is preserved, trace back to requirement that cause conflicts, and the user is asked to decide rather than automatically choosing or merging them.

# Methodology

---

# Experimental Design: Two Stages of the Master-Prompt Lifecycle

**Master-Prompt Construction** *“Can a traceable master prompt be constructed from an evolving prompt history?”*

- **Input:** 2 project histories
- **Process:** 4 consolidation strategies  $\times$  3 AI coding agents
- **Output:** candidate master prompts
- **Evaluation:** requirement-level traceability (AlignScore + untraceable rate)

**Semantic Conflict Handling** *“Can later semantic conflicts be surfaced without silently changing a settled master prompt?”*

- **Input:** hand-written ground-truth master prompt as settled specification + later prompts
- **Process:** 3 conflict-handling conditions  $\times$  the same 3 AI coding agents
- **Output:** updated specification / conflict-handling behavior
- **Evaluation:** qualitative review (RQ3)

**Experiment 1 uses ground truth as a scoring reference. Experiment 2 reuses it as the settled starting specification.**

# Two Controlled Project Histories and Ground Truth

| Project | Prompts | Description                                                                                                 |
|---------|---------|-------------------------------------------------------------------------------------------------------------|
| Simple  | 12      | Portal with navigation, conditional styling, and contact form                                               |
| Hard    | 45      | ERP-style application with customers, orders, invoices, roles, and permissions; more revisions and removals |

## GROUND TRUTH

Manually constructed from the ordered history: retains only requirements still valid at the end; later modifications replace earlier values; removals/undo remove the affected requirement; unsupported details are excluded.

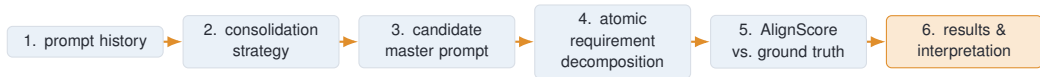
For Experiment 1, the ground-truth master prompt is also manually decomposed into atomic requirements.



# Experiment 1: Traceability Evaluation

---

# Experiment 1 Pipeline



**2 histories × 3 agents × 4 strategies = 24 candidates.**

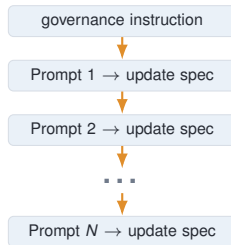
# Four Consolidation Strategies

**Group A — Single-pass**  $P_1 \rightarrow \dots \rightarrow P_N \rightarrow$   
consolidate once

- **Basic** — consolidate the conversation once at the end.
- **Loose** — additionally account for modifications, removals, and missing/additional features.
- **Strict** — additionally require every requirement to be traceable to the prompt history.

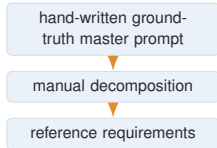
**Group B — Per-turn**  $P_1, P_2, \dots, P_N \rightarrow$   
update spec after each

- **Spec-guided** — before the prompt history begins, give the agent a governance instruction to maintain `prompts.md` and `spec.md`; after each new prompt, update the current specification.



# Challenge: Defining the Evaluation Unit

## Ground truth



## Candidate



**Example:** “Add 4 navigation buttons: Dashboard, Policies, History, Contact.” is kept as **one** requirement, not split into four — splitting would lose the four-button-set constraint.

**Atomic means a meaningful software-requirement unit, not the smallest possible sentence.**

# Why AlignScore Is Needed for Traceability

## Pair 1 — Contradiction

### GROUND TRUTH

“A navigation label starting with H must have a blue background.”

### CANDIDATE

“A navigation label starting with H must have a red background.”

BERTScore (P): **0.98**    AlignScore: **0.0002**

## Pair 2 — Faithful paraphrase

### GROUND TRUTH

“Implement customer management with create, edit, delete, and browse operations.”

### CANDIDATE

“Include customer management: Users can create, edit, delete, and browse customers.”

BERTScore (P): **0.63**    AlignScore: **0.87**

`context = ground-truth master prompt,    claim = candidate requirement`

**Similar wording can still contradict the requirement, while different wording can express the same requirement.**

# RQ1 — Effect of Consolidation Instruction Strictness

| Project | Agent  | basic       | loose       | strict             |
|---------|--------|-------------|-------------|--------------------|
| simple  | Claude | 0.086 (92%) | 0.824 (10%) | <b>0.988 (0%)</b>  |
| simple  | Codex  | 0.387 (62%) | 0.857 (18%) | <b>0.828 (18%)</b> |
| simple  | Gemini | 0.083 (97%) | 0.737 (21%) | <b>0.917 (10%)</b> |
| hard    | Claude | 0.382 (62%) | 0.362 (67%) | <b>0.905 (3%)</b>  |
| hard    | Codex  | 0.787 (19%) | 0.772 (21%) | <b>0.860 (10%)</b> |
| hard    | Gemini | 0.391 (68%) | 0.414 (60%) | <b>0.865 (10%)</b> |

- Strict > Basic in 6/6 comparisons; Strict > Loose in 5/6.
- Improvement from Basic → Loose → Strict is not always guaranteed (e.g. simple project, Codex).

**The explicit rule for traceability matters more than simply adding more instructions.**

## RQ2 — Spec-Guided vs. Single-Pass Consolidation

| Project | Agent  | strict             | spec-guided       |
|---------|--------|--------------------|-------------------|
| simple  | Claude | 0.988 (0%)         | <b>0.989 (0%)</b> |
| simple  | Codex  | 0.828 (18%)        | <b>0.987 (0%)</b> |
| simple  | Gemini | 0.917 (10%)        | <b>0.989 (0%)</b> |
| hard    | Claude | <b>0.905 (3%)</b>  | 0.882 (11%)       |
| hard    | Codex  | 0.860 (10%)        | <b>0.895 (8%)</b> |
| hard    | Gemini | <b>0.865 (10%)</b> | 0.785 (17%)       |

**Spec-guided improves traceability over the same agent's strict baseline in 4 of 6 comparisons.** All three agents improve on the simple project; the hard project is mixed. Thus, spec-guided can improve traceability, but not consistently across all projects and agents.

# Example: Non-Traceable Content in Single-Pass Consolidation

## CODEX · SIMPLE PROJECT · BASIC

“Create a **responsive static** portal homepage **using HTML, CSS, and JavaScript.**”

“Build a **modern** portal homepage, **not a marketing landing page.**”

“**Do not include a dark theme.**”

“Make sure the page works well on mobile screens: **top navigation should wrap or stack cleanly; content should collapse into single-column layouts; avoid text overflow or layout overlap on narrow screens.**”

“Keep the design polished, clean, and portal-like with **dashboard sections, quick actions, status panels, announcements,** and a usable first screen.”

**AlignScore: 0.387 · Untraceable rate: 62%** (10 of 16 requirements).



# Example: Spec-Guided Consolidation

## CODEX · SIMPLE PROJECT · SPEC-GUIDED

“Create a portal homepage. Use top navigation with 4 navigation buttons: Dashboard, Policies, History, Contact; style navigation: rounded buttons, blue accent, modern font. If the label of the navigation button starts with ‘H’, the background color of the button should be blue; if the label starts with ‘O’, the background color should be red; if the label starts with ‘D’, the background color should be ‘yellow’. Add a contact form with name, email, and message fields. The email field must be validated before the form can submit. Make sure the page also works well on mobile screens.”

- Never mentions dark theme or sidebar — both were removed. Removed requirements are excluded from the final specification.

**AlignScore: 0.987 · Untraceable rate: 0%.**

# Challenge: Updating Specification Per Turn

**Problem** — keeping `spec.md` a self-contained, current specification

- **Edited wording** can remain in `spec.md`: The earlier prompt is “Add 3 navigation buttons: Home, Orders, Contact” and then later “Change navigation buttons: Dashboard, Policies, History, Contact” → should become “Add 4 navigation buttons: . . .”
- **Conversation-like wording** can remain in `spec.md`: “I’d like to build a simple ERP web application for a small business” → should become “Build a simple ERP web application for a small business”

## WHY IT HAPPENS

Per-turn updates can preserve the wording of the latest instruction instead of reconstructing the final requirement.

### DESIGN RESPONSE

Reconstruct the affected topic from its relevant prompt history instead of patching the previous specification sentence.

# Experiment 1 — Interpretation and Trade-Off

## What worked

- Explicit traceability rules improve single-pass consolidation.
- Strict outperforms Basic in 6/6 comparisons.

## What is mixed

- Spec-guided improves in 4/6 comparisons.
- Strong on the simple project, mixed on the hard project.

## Why

Single-pass → one large reconstruction decision

Spec-guided → many smaller interpretation decisions

The 45-prompt hard project contains more revisions and removals than the 12-prompt simple project, so repeated per-turn updates create more opportunities for interpretation errors.

**Spec-guided should be understood as an alternative workflow, not a guaranteed improvement.**

## Experiment 2: Semantic Conflict Handling

---

# Experiment 2 Pipeline



**5 semantic conflicts + 2 compatible controls.**

# What Counts as a Semantic Conflict?

**Settled master prompt:** “Invoice should have a due date **15 days** after they are created. . .”

## Clear Revision

### EXAMPLE

“Change the due date to 30 days.”

### MEANING

The user replaces the settled 15-day value.

### ACTION

Update the requirement.

## Semantic Conflict

### EXAMPLE

“Add a reminder before the 30-day due date.”

### MEANING

Assumes 30 days instead of settled 15-day value.

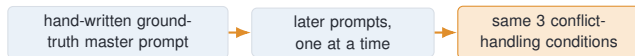
### ACTION

Keep the settled requirement and report the conflict for users.

**Semantic conflict = an incompatible later assumption without a clear revision request.**

# Experiment 2: Controlled Prompts Setup

## Starting point



### Hard project — 4 semantic conflicts

- **Due-date value** — 15-day due date vs. assumed 30-day due date
- **Recycle-bin scope** — vendors-only recycle bin vs. customer recycle-bin tab
- **Navigation layout** — top navigation bar vs. collapsible left sidebar
- **Permission** — staff cannot delete products vs. audit entry for staff deletions

### Simple project — 1 semantic conflict

- **Navigation count** — four navigation buttons vs. five buttons wrapping onto two rows

### Compatible controls

- Hard: print button
- Simple: footer note

**All agents and conditions receive the same settled specification and the same later prompts within each project.**

# Experiment 2: Conditions and Evaluation

## Three Conditions

- **Consolidate-then-ask** — consolidate first, then inspect conflicts afterward.
- **Do-not-resolve** — do not choose or merge conflicting values; keep the conflict separate.
- **Spec-guided** — check each later prompt against the settled specification as it arrives.

## Manual Evaluation

- Detect the conflict
- Do not resolve it automatically
- Keep the settled requirement
- Do not invent new details
- Show both sources

**Compatible additions should be accepted normally.**



# Gemini Output: Consolidate-Then-Ask

**Settled:** “The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”

**Later:** “Make sure all five navigation buttons wrap onto two rows on small screens.”

**Later:** “Add a footer at the bottom of the page showing a copyright notice.”

**Output: Master prompt** — “Create a portal homepage. Add top navigation. The navigation must contain **five navigation buttons**, including Dashboard, Policies, History, and Contact. If the label of a navigation button starts with ‘H’, its background color should be blue; if the label starts with ‘O’, its background color should be red; if the label starts with ‘D’, its background color should be yellow. Style the navigation with rounded buttons, a blue accent, and a modern font. Make sure all **five navigation buttons wrap onto two rows** on small screens. Add a contact form with name, email, and message fields. The email field must be validated before the form can submit. Make sure the page also works well on mobile screens. Add a footer at the bottom of the page showing a copyright notice.”

## Unresolved conflict

- **Settled / Initial Requirement (Prompt 1):** “The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”
- **Later Instruction (Prompt 2):** “Make sure all five navigation buttons wrap onto two rows on small screens.”
- **Conflict Analysis:** a navigation bar cannot contain both four and five buttons at once — the later instruction raises the count from four to five.

# Gemini Output: Do-Not-Resolve

**Settled:** “The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”

**Later:** “Make sure all five navigation buttons wrap onto two rows on small screens.”

**Later:** “Add a footer at the bottom of the page showing a copyright notice.”

**Output: Master prompt** — “Create a portal homepage. Add top navigation. If the label of a navigation button starts with ‘H’, its background color should be blue; if the label starts with ‘O’, its background color should be red; if the label starts with ‘D’, its background color should be yellow. Style the navigation with rounded buttons, a blue accent, and a modern font. Make sure navigation buttons wrap onto two rows on small screens. Add a contact form with name, email, and message fields. The email field must be validated before the form can submit. Make sure the page also works well on mobile screens. Add a footer at the bottom of the page showing a copyright notice.”

**Unresolved conflict** — Number of Navigation Buttons & Button List

- **Prompt 1:** “The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”
- **Prompt 2:** “Make sure all five navigation buttons wrap onto two rows on small screens.”

# Gemini Output: Spec-Guided Conflict Handling

**Settled:** “The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”

**Later:** “Make sure all five navigation buttons wrap onto two rows on small screens.”

**Later:** “Add a footer at the bottom of the page showing a copyright notice.”

**Output: Master prompt** — Create a portal homepage with top navigation, a contact form, and a footer.

- **Navigation Styling & Behavior:** rounded buttons, blue accent, modern font; H→blue, O→red, D→yellow; buttons wrap onto two rows on small screens.
- **Contact Form:** name, email, and message fields; email validated before the form can submit.
- **Footer:** copyright notice at the bottom of the page.
- **Responsiveness:** page works well on mobile screens.

**Unresolved conflict** — Number of Navigation Buttons

- **Settled requirement:** “The navigation must contain four buttons: Dashboard, Policies, History, and Contact.”
- **New prompt instruction:** “Make sure all five navigation buttons wrap onto two rows on small screens.”
- **Why:** the initial prompt explicitly specifies four buttons, while a subsequent prompt specifies wrapping “all five navigation buttons” onto two rows without explicitly revising the button count or naming a fifth button.

# Experiment 2: Interpretation and Key Challenge

## Interpretation

- **Consolidate-then-ask** — the conflicting assumption can enter the Master Prompt before it is checked.
- **Do-not-resolve** — keeps the conflicting value outside the Master Prompt.
- **Spec-guided** — checks the later instruction before updating the settled specification, preserves the settled value, and asks the user.

### KEY CHALLENGE — REVISION OR CONFLICT?

A later prompt may introduce a new value without making it clear whether the user intended to revise the settled requirement.

### DESIGN RESPONSE

If the revision is unclear, preserve the settled requirement and ask the user.

**The main difference is when the conflict is checked: spec-guided checks the new instruction against the settled Master Prompt before changing it.**

## Conclusion

---

# Conclusion: Answers to the Research Questions

- **RQ1** — Explicit traceability constraints reduce non-traceable content in single-pass consolidation.
- **RQ2** — Spec-guided improves 4/6 comparisons, but replaces one large reconstruction with repeated interpretation decisions.
- **RQ3** — The qualitative conflict study shows that a settled specification can be used to surface incompatible later assumptions without silently resolving them.

## TAKEAWAY

A master prompt should be treated as a maintained project specification, not merely as a one-shot summary of prompt history.

# Thank you

---

Time For Questions